Regularization and Stability

Sofia Cipolloni

October 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Regularization is a way to prevent a model from fitting the noise in the training data (overfitting). Instead of minimizing only the training error, the algorithm adds to it a penalty on the size of the weights w, multiplied by a parameter $\lambda \geq 0$ that controls its strength (the exact formulation is given in Section 2). With $\lambda = 0$ the penalty disappears and ordinary least squares is recovered; as λ grows, the weights are pushed towards zero and the model becomes simpler, up to the point where it underfits. A larger λ therefore increases the training error, but makes the model less sensitive to noise, so the best test error is typically reached at an intermediate value. As discussed in the next sections, regularization also makes the algorithm more **stable**, which means that its output changes only slightly for a small perturbation of the training set (one training point removed or replaced).
Based on this idea, Bousquet and Elisseeff [1] prove two results in their paper *Stability and Generalization* (Journal of Machine Learning Research, 2002), which is the reference of this work:

- stable algorithms generalize: the gap between test and training error is bounded in terms of a stability parameter
- algorithms that minimize a regularized empirical risk, such as ridge regression, become more stable as the regularization strength or the number of training points increases

Taken together, they explain regularization as a way to make the algorithm less sensitive to individual training points, instead of only a simplification of the model.

The goal of this project is to study this link empirically. To do so, the experiments that will be carried out aim to answer three questions:

Q1 Does stronger regularization make the algorithm more stable?

Q2 Do more stable models generalize better, that is, do they have a smaller gap between test and training error?

Q3 Is the regularization strength with the best test error also the one with the highest stability?

The first two questions follow directly from the paper, while the last one is added here because its answer turns out to be interesting: it shows whether stability alone is enough to choose the regularization strength.

Further extensions concern the size of the training set, the dimensionality of the data and the type of penalty (Ridge versus Lasso); only the first one is studied here (Section 6).

2 Theoretical basis

This section recalls the definitions and the two theorems of the paper that are needed for the study: the first relates generalization to stability, the second stability to regularization.

Let Z be the set of the possible examples $z = (x, y)$, m the training set size (n in the experiments), $S = \{z_1, \ldots, z_m\}$ a training set of i.i.d. examples $z_i = (x_i, y_i)$, A_S the function returned by an algorithm A trained on S, and ℓ a loss. The *risk* $R(A_S) = \mathbb{E}_z[\ell(A_S, z)]$ cannot be computed, while the *empirical risk* is $R_{\text{emp}}(A_S) = \frac{1}{m} \sum_i \ell(A_S, z_i)$.
A model *generalizes* well when it performs similarly on seen and unseen data, so if the ***generalization gap*** $R - R_{\text{emp}}$ (estimated by the difference between the test error and the training error) is small. By contrast, a large gap indicates overfitting: the model has fitted the training data, including its noise, and performs much worse on new examples.

Uniform stability. Let $S^{\setminus i}$ be S without its i-th example. The algorithm A has *uniform stability* β with respect to the loss ℓ if

$$\forall S \in Z^m,\ \forall i \in \{1,\dots,m\},\ \forall z \in Z: \qquad \big|\,\ell(A_S, z) - \ell(A_{S^{\setminus i}}, z)\,\big| \ \le\ \beta.$$

A smaller β means that removing any training point does not significantly change the loss, and therefore the algorithm is more stable.

The paper also shows that **stability implies generalization** (Theorem 12 in [1]) as the generalization gap of a uniformly stable algorithm is controlled by β. For a bounded loss $\ell \le M$, with probability at least $1-\delta$,

$$R(A_S)\ \le\ R_{\text{emp}}(A_S) + 2\beta + \big(4m\beta + M\big)\sqrt{\frac{\ln(1/\delta)}{2m}}.$$

The bound is informative only if β decreases faster than $1/\sqrt{m}$, and it does not depend on the size of the class of predictors, but only on the behavior of the algorithm.

Regularization, stability and expected behavior. Consider regularized empirical risk minimization, which for a linear predictor is

$$A_S = \arg\min_w\ \frac{1}{m}\sum_{i=1}^{m} \ell(w, z_i) + \lambda\|w\|_2^2$$

(with the squared loss it is ridge regression). Under suitable boundedness assumptions on the loss and on the data, Theorem 22 in the paper [1] proves that

$$\beta\ \le\ \frac{c}{\lambda m},$$

where c depends on the loss and on the scale of the data. The penalty makes the objective strongly convex with a constant proportional to λ, while removing one example changes the data term only by about $1/m$, so the minimizer cannot move much: as λ or m increases, β decreases and the algorithm becomes more stable.
For $\lambda \to 0$ the bound is vacuous, so it does not apply to unregularized least squares, while a larger value of the parameter raises the training error, since the predictor fits the data less closely (more bias, less variance).

In short, the theory suggests that stronger regularization (and more data) makes the algorithm more stable, which implies a smaller gap between training and test error; it does not say, however, that the most stable algorithm has the best test error. These statements are tested in practice by the experiments of Section 5. The stability measure used in the experiments is an empirical version of β (Section 4): it follows its trend but is not the same quantity.

3 Datasets

The project's goal requires both a real dataset and a synthetic one in order to compare the behavior of the algorithms on data whose generating process is unknown (real) and on data for which it is fully controlled (synthetic).

Real dataset. The Diabetes dataset from `scikit-learn` [2] is used: 442 patients, 10 clinical features (age, sex, body mass index, blood pressure and six blood serum measurements) and a target measuring the progression of the disease one year after the baseline.
Other datasets were also taken into account at the beginning of the project, but Diabetes was

chosen for three main reasons. First, it is a regression problem with a noisy target, so there is a real risk of fitting noise, which is exactly the situation in which regularization is expected to matter. The Wine Quality dataset, instead, had a target that is an integer score with only a few distinct values, which makes it closer to a classification problem with ordered classes than to a regression one, so that the squared loss is not really appropriate for it.
Second, it is small enough to run the many retrainings required by the leave-one-out stability estimate in a short time. A larger dataset such as California Housing (20,640 samples) would have made this estimate much more expensive: one refit is needed for each training point and each refit costs $O(nd^2)$, so the total cost grows roughly quadratically with n.
Finally, it is freely available, so the experiments can be reproduced by anyone without the need to download external data.

Synthetic dataset. The synthetic data are generated from a linear model

$$y = x^\top w^\star + \varepsilon, \qquad x \sim \mathcal{N}(0, I_d), \quad \varepsilon \sim \mathcal{N}(0, \sigma^2),$$

with $n = 500$ samples, $d = 20$ features and $\sigma = 1$ (since the generating process is known, the best possible test error is the noise variance $\sigma^2 = 1$).
The true weights $w^\star$ are drawn from $\mathcal{N}(0, 1)$ and then 30% of them (6 out of 20) are set to 0, so that the problem has a sparse structure.
With $n \gg d$ and low noise, this dataset represents an easy scenario, in which least squares is expected to perform already well. However, since the generating process is fully controlled, the number of samples n and features d can be varied independently, which makes it possible to move to harder regimes (fewer samples, more features) in the extensions (Section 6).

3.1 Train/test split and preprocessing

Each dataset is split once, at random, into a training set (75%) and a test set (25%) with a fixed seed that makes them exactly reproducible.
The test set is used only to evaluate the final models in order to avoid *data leakage*, the problem for which information from the test set influences the training phase, making the test error an optimistic estimate of the performance on unseen data.

For the *real dataset* two preprocessing steps are applied, and in both cases the required statistics are computed on the training set only.

- **Feature standardization:** the ridge penalty depends on the scale of the features, so they are standardized (zero mean, unit variance) using the training statistics.
- **Target centering:** the training mean of y is subtracted from all the values of the target, in both the training and the test set.

For the *synthetic dataset* no scaler is applied: by construction, all the features are independent standard normal variables, so they already have (approximately) zero mean and unit variance and they are on the same scale.

Dataset	Train / Test	Features	$\mathrm{Var}(y_{\mathrm{train}})$	$\mathrm{Var}(y_{\mathrm{test}})$
Diabetes (real)	331 / 111	10	6253	4964
Synthetic	375 / 125	20	16.7	16.8

Table 1: Summary of the datasets.

4 Method and implementation

4.1 Models

Two models are compared, both implemented from scratch with NumPy. Given the training inputs $X \in \mathbb{R}^{n \times d}$ and targets y, a column of ones is added to X to obtain $S = [\mathbf{1} \mid X]$, so that the model has an intercept w_0, which is treated as one more weight of w.

- ***Ridge regression*** minimizes $\|Sw - y\|_2^2 + \alpha\|w\|_2^2$, with $\alpha > 0$, and has the closed-form solution $w = (\alpha I + S^\top S)^{-1} S^\top y$.
  - $\alpha I + S^\top S$ is always invertible, so the solution is always unique.
  - weights are shrunk towards zero as the regularization parameter grows (for $\alpha \to \infty$, $w \to 0$).

- ***Unregularized least squares*** minimizes $\|Sw - y\|_2^2$ and has the closed-form solution $w = (S^\top S)^{-1} S^\top y$.
  - the solution is unique only if $S^\top S$ is invertible, and it can be very sensitive to small changes in the training set when $S^\top S$ is nearly singular.
  - corresponds to $\alpha = 0$ and it is not covered by the stability bound of the paper (Section 2).

Two implementation choices need to be specified:
(i) the penalty is applied to all the weights, *including the intercept*, which is harmless in practice as the target of the real dataset is centered and the one of the synthetic dataset has a mean close to zero (Section 3).
(ii) the *regularization parameter*: the strength of the penalty can be parametrized in two equivalent ways: by multiplying the *average* loss by a parameter λ, as in the paper, or by multiplying the *sum* of the squared errors by a parameter α.

$$\underbrace{\frac{1}{n}\sum_{i=1}^{n} \ell(w, z_i) + \lambda\|w\|_2^2}_{\text{average loss, parameter } \lambda} \quad ; \quad \underbrace{\sum_{i=1}^{n} \ell(w, z_i) + \alpha\|w\|_2^2}_{\text{sum of the errors, parameter } \alpha} \quad .$$

The second objective is n times the first when $\alpha = \lambda n$, so the two have the same minimizer and describe the same model.

This work uses the formula on the right as it leads directly to the closed-form solution specified above (without any rescaling by n). However, since the stability bound of the paper is stated in terms of λ (Section 2), this parameter is exploited to relate the results to the theory. This choice has two consequences. First, the sum of the errors grows with n while the penalty does not, so the same α is a weaker regularization when n is larger, and is not comparable across datasets with a different size (unlike λ). Second, when n varies, keeping a fixed α or λ is not equivalent, so both cases are considered in Section 6.

4.2 Regularization strength

The experiments are organized as a **sweep over $\boldsymbol{\alpha}$**, that is, the repetition of the same procedure for each value of the parameter in a grid, in the following phases:
1. The experiments are run over a logarithmic (as orders of magnitude differ across datasets) grid of 16 values of α: $\alpha = 0$ (unregularized least squares) and 15 log-spaced values from 10^{-3} to 10^4 — from nearly no regularization to a regularization so strong that the model is almost constant.
2. For each value of α the following metrics are computed: *training* and *test mean squared error (MSE)*, *test* $R^2 = 1 - MSE_{test}/\mathrm{Var}(y_{test})$ and *gap between test and training MSE* (an empirical proxy of the generalization gap that the stability theory bounds).

3. The regularization parameter is then selected by **5-fold cross-validation** on the training set only: the selected value is the α with the *lowest average validation mean squared error*. *Remark.* The features of the real dataset are standardized on the whole training set before the cross-validation, so the statistics of the scaler also include the validation folds; this is a mild form of leakage that stays inside the training set and never involves the test set.

4.3 Stability

Stability is measured with a **leave-one-out procedure**: for a fixed α, a reference model is trained on the whole training set and it is then retrained, with the same α, on the training set without point i ($\forall i \in \{1, \dots, n\}$).
Given T: test set, f: reference model, $f_{\setminus i}$: model without point i, the two models are compared on the same fixed test set, and the following quantities are computed:

- the *mean absolute change in the prediction*, $\frac{1}{|T|} \sum_{x \in T} |f_{\setminus i}(x) - f(x)|$
- the *mean absolute change in the squared loss*, $\frac{1}{|T|} \sum_{(x,y) \in T} \big|\ell(f_{\setminus i}(x), y) - \ell(f(x), y)\big|$, which is the closest to the definition of β (Section 2).

Both quantities
- follow the trend of β rather than estimating it: they are averages over the removed points and over a finite test set, instead of a supremum, and consider only the removal of a point, not its replacement;
- are averaged over all the removed points: the smaller the obtained value, the more stable the algorithm.

The full procedure requires $n + 1$ trainings for every α (the reference model and one model for each removed point), which is feasible for the small size of the datasets and the closed-form formula.

5 Main experiments

The main experiments study how regularization affects the stability and the generalization of ridge regression, compared with unregularized least squares, on the real and the synthetic dataset. They consist of the sweep over the grid of the regularization parameter described in Section 4, among whose values the cross-validation selects $\alpha \approx 31.6$ on the Diabetes dataset and $\alpha \approx 0.316$ on the synthetic one. As regards the former one, the mean validation MSE decreases from 2991 for least squares to 2984 at the selected value, a small improvement compared with its standard deviation across the folds ($\approx$ 657–683); on the synthetic data, instead, it is almost constant (1.235) for $\alpha \lesssim 1$, so the selected value is not meaningful in itself and indicates that regularization is not needed there. Other values are also relevant for the analysis:
- $\alpha = 0$ (unregularized least squares): reference of the whole study;
- the value with the lowest test MSE, which is only an optimistic benchmark as it uses the test set and could not be known in practice;
- a very strong regularization ($\alpha = 10^4$).

Dataset	α	Train MSE	Test MSE	R^2	Gap	Pred. change
Diabetes	0 (least squares)	2780.1	3180.2	0.359	400.1	0.329
	31.6 (CV)	**2803.3**	**3147.9**	**0.366**	**344.6**	**0.276**
	100 (lowest test)	2882.6	3082.6	0.379	200.0	0.232
	10^4	5702.1	4594.3	0.075	−1107.8	0.021
Synthetic	0 (least squares)	1.094	1.006	0.940	−0.088	0.0088
	0.316 (CV)	**1.094**	**1.006**	**0.940**	−0.088	**0.0088**
	100	1.704	1.682	0.900	−0.022	0.0085
	10^4	15.454	15.599	0.074	0.145	0.0011

Table 2: Summary of the sweep over α (whole grid in the `results/` folder of the repository). *Note:* on the synthetic dataset the lowest test MSE is reached in the flat region, practically coincident with least squares, so the intermediate value $\alpha = 100$ is reported instead.

The sweep is then analyzed in two experiments. The first (Section 5.1) checks if regularization makes the algorithm more stable, while the second (Section 5.2) how it affects the training and test errors, together with the generalization gap. Their comparison reveals whether more stable models generalize better, as predicted by the theory (Section 2).

5.1 Stability versus regularization strength

Of the two quantities defined in Section 4, Figure 1 shows the stability as the mean absolute change in the test prediction when one training point is removed, and displays it as a function of α; the mean absolute change in the loss is reported in Appendix A.

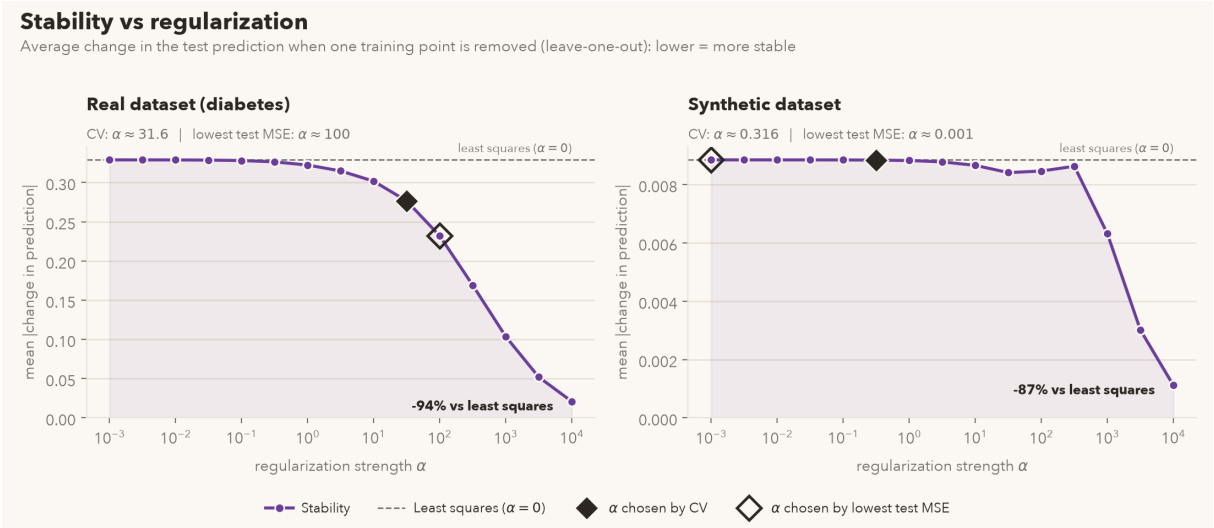


Figure 1: Stability as a function of α for the real (left) and the synthetic (right) dataset

On the real dataset, stability improves steadily as the regularization strength rises: the change in the prediction is flat up to $\alpha \approx 1$ and then presents a monotonic decline, from 0.329 for least squares to 0.276 at the value selected by cross-validation ($\alpha \approx 31.6$, -16%), 0.232 at $\alpha = 100$ (-29%) and 0.021 at $\alpha = 10^4$ (-94%).
On the synthetic dataset the curve is flat (at a value of about 0.0088) up to $\alpha \approx 300$, due to the fact that least squares is already very stable. The penalty starts to matter only when α becomes comparable with the eigenvalues of $S^\top S$, which are of order n for these data (here $n = 375$), consistently with the point at which the curve starts to decrease ($\alpha \gtrsim 300$). Then it drops steeply, to 0.0063 at $\alpha = 10^3$ and to 0.0011 at $\alpha = 10^4$ (-87%).
Hence, on both datasets applying **regularization increases stability**: there is a clear positive

relation between them, since the change in the prediction decreases as α grows, up to -94% on the real dataset and -87% on the synthetic one at $\alpha = 10^4$. This agrees with the theory, which states that the stability bound improves as the regularization parameter grows. The two datasets differ in the range of α in which the effect arises: on Diabetes it starts when the parameter is still low, while on the synthetic dataset only for higher α values, since least squares is already very stable.
Measuring stability with the change in the loss instead of the change in the prediction gives the same trend on the real dataset, but a non-monotone curve on the synthetic one (see Figure 4 and Appendix A).

5.2 Test error versus regularization strength

Figure 2 shows the training and test MSE as a function of α. The generalization gap (their difference) is also represented on the graph through a shaded area.

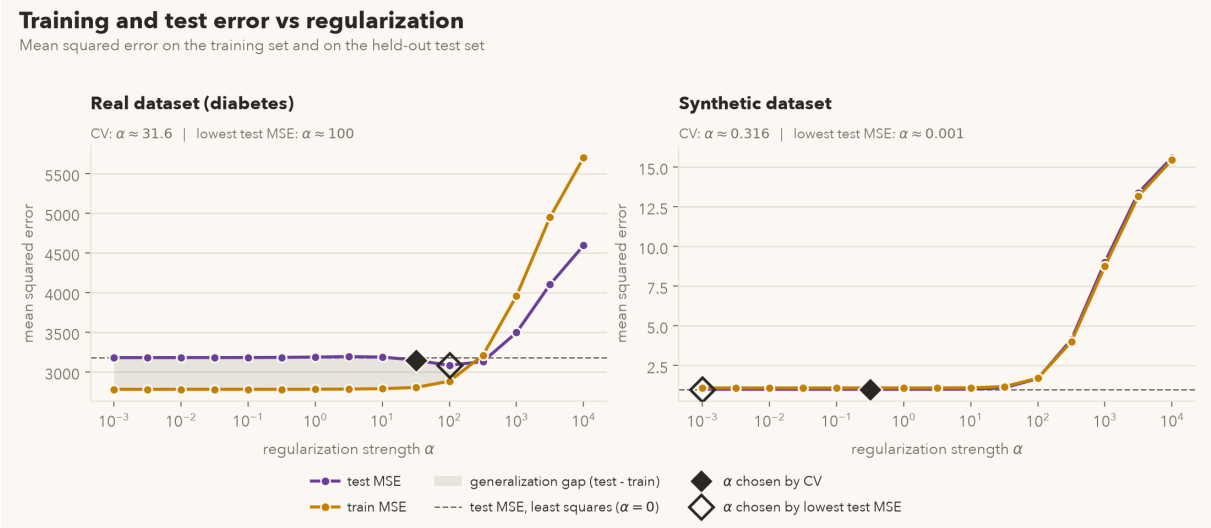


Figure 2: Training and test MSE as a function of α for the real (left) and the synthetic (right) dataset.

On the real dataset the test error has the expected U shape. The training MSE grows with α (from 2780 to 5702), since the model fits the data less closely, while the mean squared error of the test set drops from 3180 for the unregularized model to a minimum of 3083 at $\alpha = 100$ (R^2 from 0.359 to 0.379, -3.1%), and then starts to increase, becoming 4594 ($R^2 = 0.075$) at $\alpha = 10^4$ because of underfitting.
The value selected by cross-validation, $\alpha \approx 31.6$, exhibits a test MSE of 3148 ($R^2 = 0.366$, -1.0% with respect to least squares), slightly worse than the best value on the test set.
Regarding the generalization gap, it decreases from 400 for least squares to 200 at $\alpha = 100$ and becomes negative for $\alpha \gtrsim 300$, because the test target has a smaller variance than the training one (Table 1), so that a nearly constant predictor has a smaller error on the test rather than on the training set. Since the gap measures how much better the model fits the data on which it has been trained than the new ones, it shrinks when α grows as the regularized model follows the training examples less closely (in line with its higher stability). Its negative values, instead, do not indicate a good model: when α is large the predictor is almost constant, its error tends to the variance of the target on each set (Table 1), and the test set is simply easier to predict.
Overall, the behavior of the test MSE is the usual bias–variance trade-off: up to $\alpha \approx 100$ the penalty reduces the variance of the solution and the model is more stable, keeping the test MSE low; beyond this value, however, the bias introduced by shrinking the weights prevails and the model underfits, so that the test MSE increases again even though the stability keeps

improving.
On the synthetic dataset regularization does not appear to be useful: the test MSE of least squares (1.006, $R^2 = 0.940$) is already at the noise level $\sigma^2 = 1$, and it stays unchanged up to $\alpha \approx 10$, so that the value selected by cross-validation gives the same test MSE as least squares. Here $\sigma^2 = 1$ is the variance of the noise added to the targets, that is the error that no model can remove, so least squares is already only 0.6% above the best possible value and there is nearly no room for regularization to improve it.
For larger values the test MSE increases (1.10 at $\alpha \approx 31.6$, 1.68 at 100, 15.6 at 10^4, with $R^2 = 0.074$). The gap is small and negative up to $\alpha \approx 100$ and positive afterwards, with a maximum of 0.25 at $\alpha = 10^3$. For this reason, the training and the test curves of the synthetic dataset are almost overlapped for α up to about 100: with few parameters compared with the number of points ($n = 375 \gg d = 20$) the model does not overfit, so it performs equally well on both the training and the test data.
The two curves separate only for larger α, when both errors increase and the test one begins to grow faster.

6 Extension: effect of the training set size

The extension phase focuses on the number of training points n as the principal quantity to study in addition to the regularization strength, for two main reasons:

- The stability bound of the paper $\beta \leq c/(\lambda n)$ improves as either λ or n grows, so considering more data should have a similar effect to increasing regularization: this extension checks whether this is the case *in practice*.
- The main experiments are run at a single value of n, in an easy setting ($n \gg d$) where least squares is already stable and regularization brings no benefit on the test error. Varying n shows in which settings (few training points) regularization is useful and in which it is not.

This study is carried out on the synthetic dataset only: points can be drawn with the same noise level and the same number of features since they are generated by a known process, unlike in the real scenario (fixed number of examples).

Design. A pool of 4000 points is generated with the same process of Section 3 ($d = 20$, $\sigma = 1$), 3000 of which form the training set and 1000 the fixed test one. For each $n \in \{25, 50, 100, 200, 400, 800, 1600, 3000\}$ (roughly equally spaced on a logarithmic axis), the procedure follows 4 steps:
1. n points are drawn at random from the pool;
2. the model is trained on them;
3. training error, test error and stability (mean absolute change in test prediction) are measured, always on the same test set and in the same way as in the main experiments, except for one fact: the leave-one-out is run on a sample of 100 randomly chosen points only, as its average will still represent a good estimate but requires a lower running time;
4. the previous steps are repeated with 5 different random subsets and the average of the results is computed, so that they do not depend on the luck of a single subset.
Since the test set never changes, the differences between runs depend only on n and on the regularization.

Fixing the regularization. Two ways of choosing the regularization while n grows are compared, which correspond to the distinction between α and λ of Section 4. Recall that the ridge solution minimizes

$$\sum_{i=1}^{n}\big(y_i-\hat{y}_i\big)^2+\alpha\|w\|^2 \;=\; n\left[\frac{1}{n}\sum_{i=1}^{n}\big(y_i-\hat{y}_i\big)^2+\lambda\|w\|^2\right] \qquad \text{with } \alpha=\lambda n$$

(i) ***fixed*** $\boldsymbol{\alpha}$ ($\alpha \in \{0, 1, 10, 100, 1000\}$). Since the loss is summed over the training points, the sum of the squared errors grows with n while the penalty $\alpha\|w\|^2$ does not, so it weighs less and less with respect to the data as n grows: any gain in stability comes from the larger sample alone.
(ii) ***fixed*** $\boldsymbol{\lambda}$ ($\lambda \in \{0, 0.1, 1, 10\}$, with $\alpha = \lambda n$). As the right-hand side of the formula shows, the loss is averaged and the relative weight of the penalty stays the same as n grows. This is the setting of the paper, where the bound $\beta \le c/(\lambda n)$ depends on both λ and n.
In both cases the value 0 is least squares.



Figure 3: Stability (top) and test MSE (bottom) as a function of the training set size n for the synthetic dataset, with α fixed (left) and λ fixed (right). The dashed line has slope -1 (a trend proportional to $1/n$) and the dotted line is the noise variance, the best possible test error. Both axes are logarithmic.

Results. Stability improves with n for all the models, and roughly as $1/n$ from $n \approx 100$ on, as shown by the dashed line in Figure 3: the change in the prediction of least squares goes from 1.02 for $n = 25$ to 0.0010 for $n = 3000$. Test error and generalization gap also get better: for least squares, the former diminishes from 7.6 towards the noise variance (1.07 for $n = 3000$), and the latter from 7.4 to 0.08.
It emerges that the two methods of regularizing behave differently.

With a fixed α, the advantage of regularization is large for small training set size and disappears as it grows. At $n = 25$ (close to d), $\alpha = 1000$ makes the stability 83 times better than the unregularized benchmark (0.012 versus 1.02); the gain decreases to 3.8 times at $n = 100$ and 1.4 times at $n = 400$, and vanishes for $n \geq 800$. The test error is not improved by such a strong penalty: it is higher than that of least squares for every $n \leq 1600$ (for instance 15.7 versus 7.6 at $n = 25$). A smaller amount of regularization helps where it matters: at $n = 25$, $\alpha = 1$ reduces the test MSE from 7.6 to 2.8 and the instability from 1.02 to 0.35, while for $n \geq 200$ all the values of $\alpha \leq 10$ are equivalent to the unregularized model. This is consistent with the theory: with a fixed α the bound $c/(\lambda n) = c/\alpha$ does not depend on n, so it gives no guarantee of improvement when n grows.

With a fixed λ, the stability curves are parallel to that of least squares, so the advantage of regularization stays constant instead of vanishing, and its size depends on λ: for $\lambda = 0.1$ and $\lambda = 1$ the improvement is negligible, while for $\lambda = 10$ stability turns out to be 3 times better than the baseline for every $n \geq 200$. This strong regularization, however, underfits: it shrinks the weights too much, and since λ is fixed this bias does not decrease as n grows, so that the test MSE is about 13.8 for all the values of n, against a best possible value of 1. In other words, the $1/n$ trend of the bound is reproduced by the observed stability, but it also appears without regularization, so it cannot be attributed to the penalty alone.
In conclusion, it can be stated that a fixed α loses its effect on stability as n grows, a fixed λ keeps it constant, and in both cases a stronger penalty makes the algorithm more stable but can worsen the test error.

7 Discussion

The results are discussed with respect to the three questions of the Introduction.

Does stronger regularization make the algorithm more stable? *Yes, on both datasets.* The change in the prediction decreases with the regularization parameter (-94% on Diabetes and -87% on the synthetic data at $\alpha = 10^4$), in agreement with the bound $\beta \lesssim c/(\lambda n)$. The effect starts at $\alpha \approx 1$ on Diabetes, but only at $\alpha \gtrsim 300$ on the synthetic data, where $n \gg d$ and least squares is already very stable. Moreover, the conclusion on stability depends on the metric: measured on the loss, it is not monotone on the synthetic data (Appendix A).

Do more stable models generalize better? *Only in part.* On Diabetes the gap shrinks as stability grows, but the test error deteriorates because of underfitting beyond $\alpha \approx 100$: the theory holds for the former, not for the latter. On the synthetic data the difference between test and training error is already almost null for least squares and even widens for large α, where the model underfits: the bound is only an upper bound on the gap, so a lower cap does not imply a smaller discrepancy. The extension on n is closer to the prediction: the gap and the instability decrease together as n grows (Section 6).

Is the best regularization also the most stable one? *No.* The test error is at its highest at the strongest penalty, where stability also peaks, and it is lowest at an intermediate value of the regularization parameter ($\alpha = 100$ on Diabetes, where stability is only 29% better than least squares). On the synthetic data the test error has no interior minimum and regularization is useless, in line with the extension: the benefit shows up with a low n (at $n = 25$, $\alpha = 1$ reduces the test MSE from 7.6 to 2.8) and fades as it grows. The value chosen by cross-validation lowers the test error of least squares by only 1% on Diabetes, within the variability across folds, and is not meaningful on the synthetic data.

Note: the comparison between the datasets is made on the trends: while the relative changes in stability and the R^2 are comparable (they do not depend on the scale of the target), the values of α are not.

Limitations. The conclusions have to be read with some care, because of a few limitations emerged during the analysis and already mentioned in the previous sections.

- The main results come from a single train/test split, and the real test set contains only 111 points: differences of a few percentage points in the test error cannot be considered significant, while the trend of stability, much larger, is robust.
- The stability measure is an empirical average over the removed points and a finite test set, and it only considers the removal of a point; it is not the supremum used in the paper, and different metrics (prediction or loss) can behave differently.
- The theorems require bounded losses and data, which holds only approximately for the Diabetes target and not for Gaussian noise, and they do not cover unregularized least squares ($\lambda = 0$).
- The features of the real dataset are standardized before the cross-validation, causing a mild leakage that involves only the training set.
- Only two datasets and one extension (the training set size) have been studied; the effect of the dimension and of the type of penalty (L1 versus L2) is left for future work.

8 Conclusions

This project studied empirically the relationship between regularization, stability and generalization of ridge regression, as in the paper by Bousquet and Elisseeff. The experiments reveal that increasing the regularization makes the algorithm more stable, in agreement with the theory, with a very large effect on the real data. Stability, however, is not enough to explain the test error, which reaches its minimum at a middle penalty and worsens when the model becomes too stable, and on an easy dataset regularization brings no benefit.
The generalization gap and stability improve together as the number of training points grows, and the advantage of regularization over least squares is largest for few points and vanishes as n increases. The main limitations are the use of a single split on small datasets and of an empirical version of stability; future work could repeat the experiments on several splits, study the dimension of the data and the L1 penalty, and measure the stability with the replacement of a point as in the paper.

A Stability measured with the change in the loss

Figure 4 shows the stability measured as the mean absolute change in the squared loss on the test set, instead of the change in the prediction (compare with Figure 1).

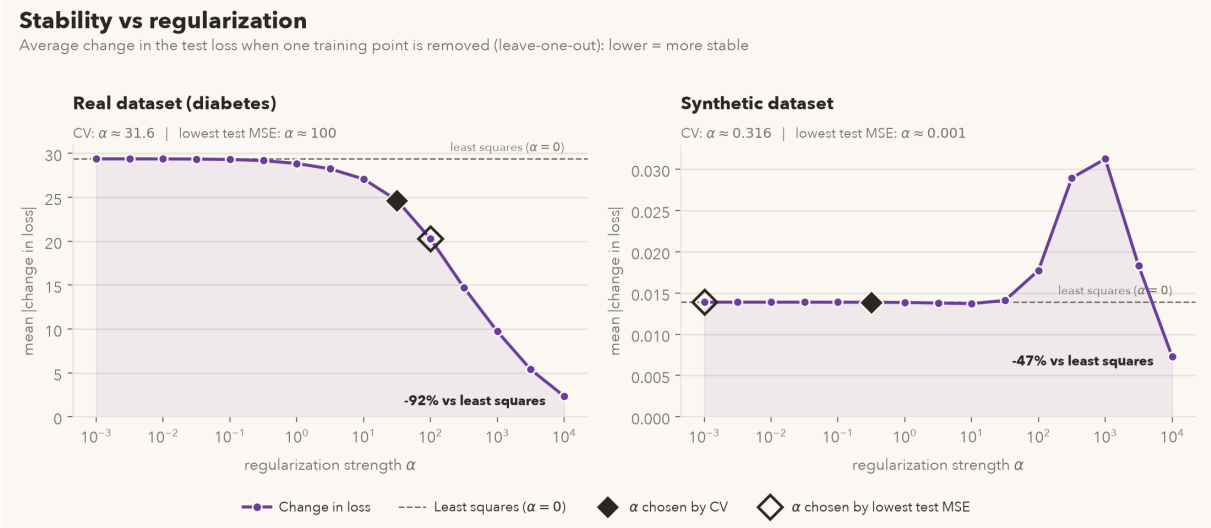


Figure 4: Stability (now mean absolute change in the squared loss on the test set) as a function of α for the real (left) and the synthetic (right) dataset.

On the real dataset the variation of the loss decreases monotonically with α, like the shift of the predictions, whereas on the synthetic dataset it is not monotone. Being roughly the product of the residual size and that shift, this quantity also depends on the errors of the model: on the synthetic dataset they grow strongly with α (the test MSE rises from about 1 to about 15), and up to $\alpha \approx 10^3$ this growth outweighs the decrease of the shift, which creates a peak; on the real dataset the errors grow only moderately compared with the fall of the shift, so no maximum appears.

References

[1] O. Bousquet and A. Elisseeff. Stability and Generalization. *Journal of Machine Learning Research*, 2:499–526, 2002.

[2] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.